

Analysis Modeling and Data Flow Diagrams (DFDs)

Module 1 | IIIT Nagpur
(Textbook-level, Exam-Oriented)

Role of Analysis Modeling

- Analysis modeling is concerned with understanding WHAT the system should do, independent of implementation.
 - It represents user requirements in a structured, visual form to reduce ambiguity.
 - Models act as a bridge between requirements and architecture.
 - They support early evaluation of design alternatives.
-

Need for Analysis Models

- Natural language requirements are often incomplete, inconsistent, and ambiguous.
- Models provide a common language between users, analysts, and architects.
- Early modeling reduces development cost by identifying errors early.

Errors early or identified

A traditional analysis is a structured analysis

Traditional Analysis Models

- Structured analysis uses graphical models to represent system behavior.
- The most widely used models are Data Flow Diagrams (DFDs) and Entity Relationship Diagrams (ERDs).
- This lecture focuses on DFDs as a primary functional modeling technique.

What is a Data Flow Diagram (DFD)?

- A Data Flow Diagram models the flow of information through a system.
- It shows how data is transformed by processes and stored in data stores.
- DFDs focus on logical data movement, not control flow or timing.

Purpose of DFDs

- 
- To understand system functionality from a data perspective.
 - To identify major system processes and data dependencies.
 - To support communication with both technical and non-technical stakeholders.

easy to learn

Basic DFD Notation

- DFDs are constructed using only four basic symbols.
- These symbols have simple rules, making DFDs easy to learn and use.
- Consistency of notation is essential for correct interpretation.

DFD Element: Process (Definition)

- A process represents an activity that transforms input data into output data.
- A process exists only if data is changed or manipulated.
- Processes are always placed inside the system boundary.

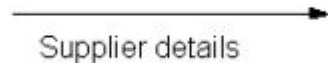


Process Naming Rules

- Process names should be verb–noun phrases such as 'Validate Order' or 'Record Payment'.
- Generic names such as 'Process Data' should be avoided.
- Each process must have at least one input and one output.

DFD Element: Data Flow

- A data flow represents the movement of information between components.
- Data flows are shown as labeled arrows.
- Data must always flow to or from a process.

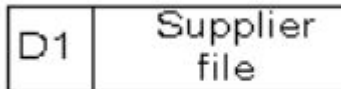


Data Flow Rules

- Data cannot flow directly between two data stores.
- Data cannot flow directly between two external entities.
- Each data flow must be named using a noun or noun phrase.

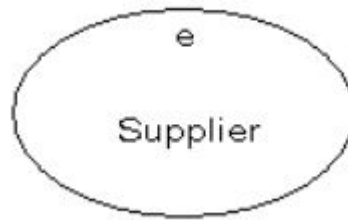
DFD Element: Data Store

- A data store represents a repository where data is stored for later use.
- It may represent a file, database, or physical storage.
- Each data store must have at least one input and one output.



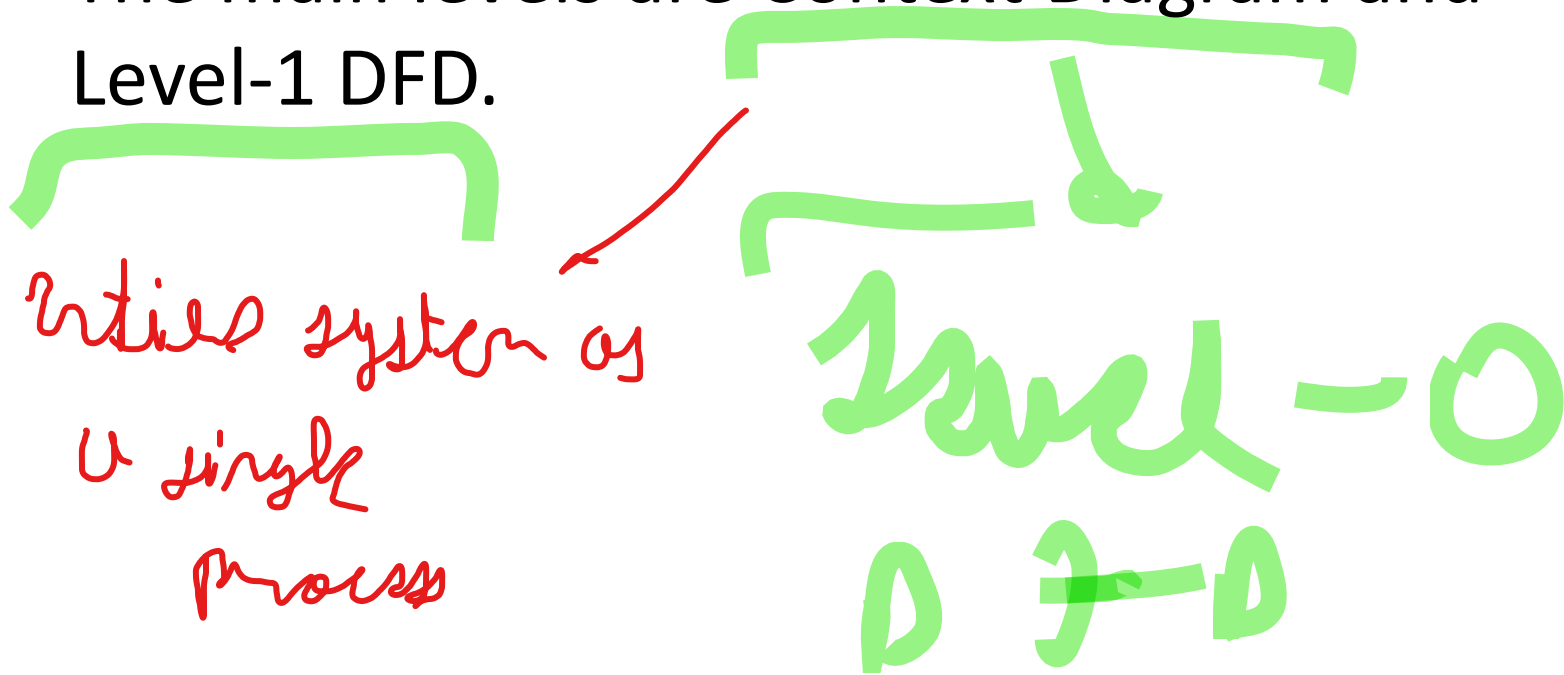
DFD Element: External Entity

- An external entity represents a source or destination of data outside the system.
- External entities define the system boundary.
- Examples include users, customers, suppliers, or external systems.



Levels of Data Flow Diagrams

- DFDs are developed in multiple levels to manage complexity.
- Each level provides **increasing** detail.
- The main levels are Context Diagram and Level-1 DFD.



Context Diagram (Level 0 DFD)

- A context diagram represents the entire system as a single process.
- It shows all external entities and major data flows.
- It defines the scope and boundary of the system.

Purpose of Context Diagram

- To clearly define what is inside and outside the system.
- To identify external interfaces.
- To establish agreement with stakeholders.

Level 1 Data Flow Diagram

- A Level 1 DFD decomposes the system into major sub-processes.
- It shows internal data flows and data stores.
- It provides a more detailed functional view.

Level-2 DFD

- A Level-2 Data Flow Diagram is a further decomposition of a Level-1 DFD process.
- It shows the internal working of a single process in greater detail.
- Level-2 DFDs are used when Level-1 processes are complex.

Use when level 1 processes are complex

Rules for Level-2 DFD

- Only one Level-1 process is expanded.
- Process numbering must follow hierarchy.
- No new external entities are introduced.
- Balancing with Level-1 DFD must be maintained.

Ensures consistency between different levels of DFD

Balancing in DFDs

Q. Why is balancing needed in DFD?

- Balancing ensures consistency between different levels of DFDs.
- All inputs and outputs of a process must appear unchanged in its decomposition.
- Balancing prevents loss or creation of data.

Functional Decomposition

Complex → simple sub processes

- Functional decomposition breaks complex processes into simpler sub-processes.
- It improves **clarity** and **understanding**.
- Decomposition continues until processes are simple enough.

Process Numbering

- Processes are numbered hierarchically to indicate decomposition.
- Example: Process 3 decomposes into 3.1, 3.2, 3.3.
- Numbering does not indicate execution order.

Case Study Introduction: Video Rental System

- The system rents videos to customers.
- It maintains customer records, stock, and transactions.
- The system interacts with customers and suppliers.

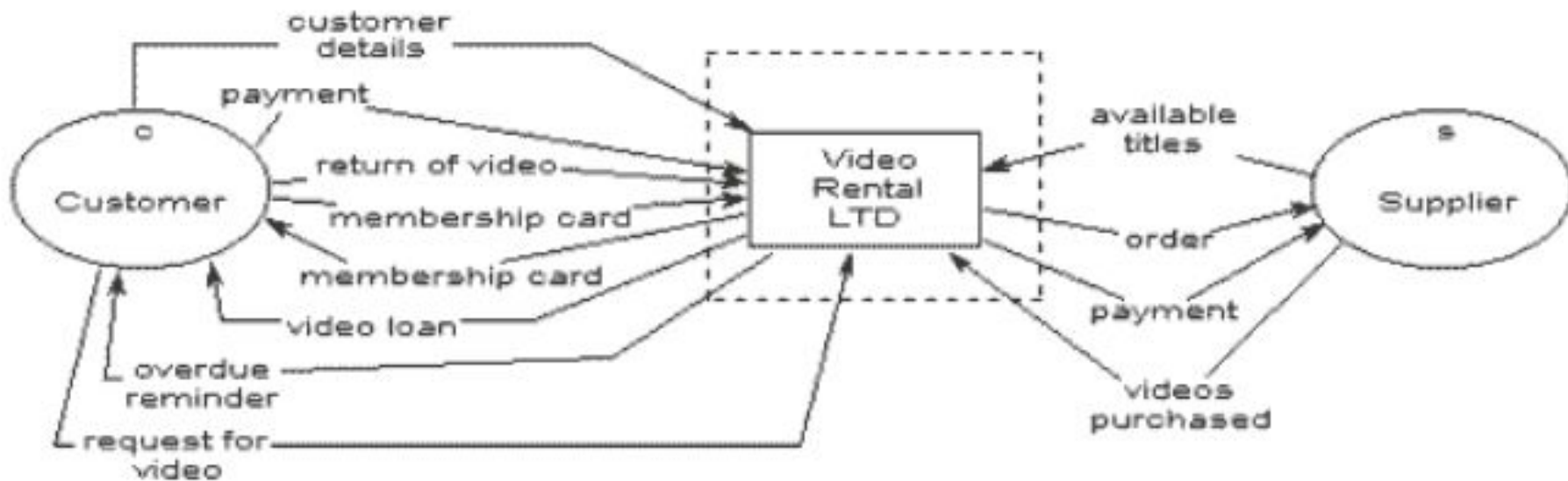
Identifying External Entities (Case Study)

- Customer: rents and returns videos.
- Supplier: provides new video stock.
- These entities lie outside the system boundary.

Context Diagram – Case Study

- The entire video rental system is shown as a single process.
→ or a single process
- Major data flows include rental request, payment, and reminders.
- External entities interact with the system via these flows.

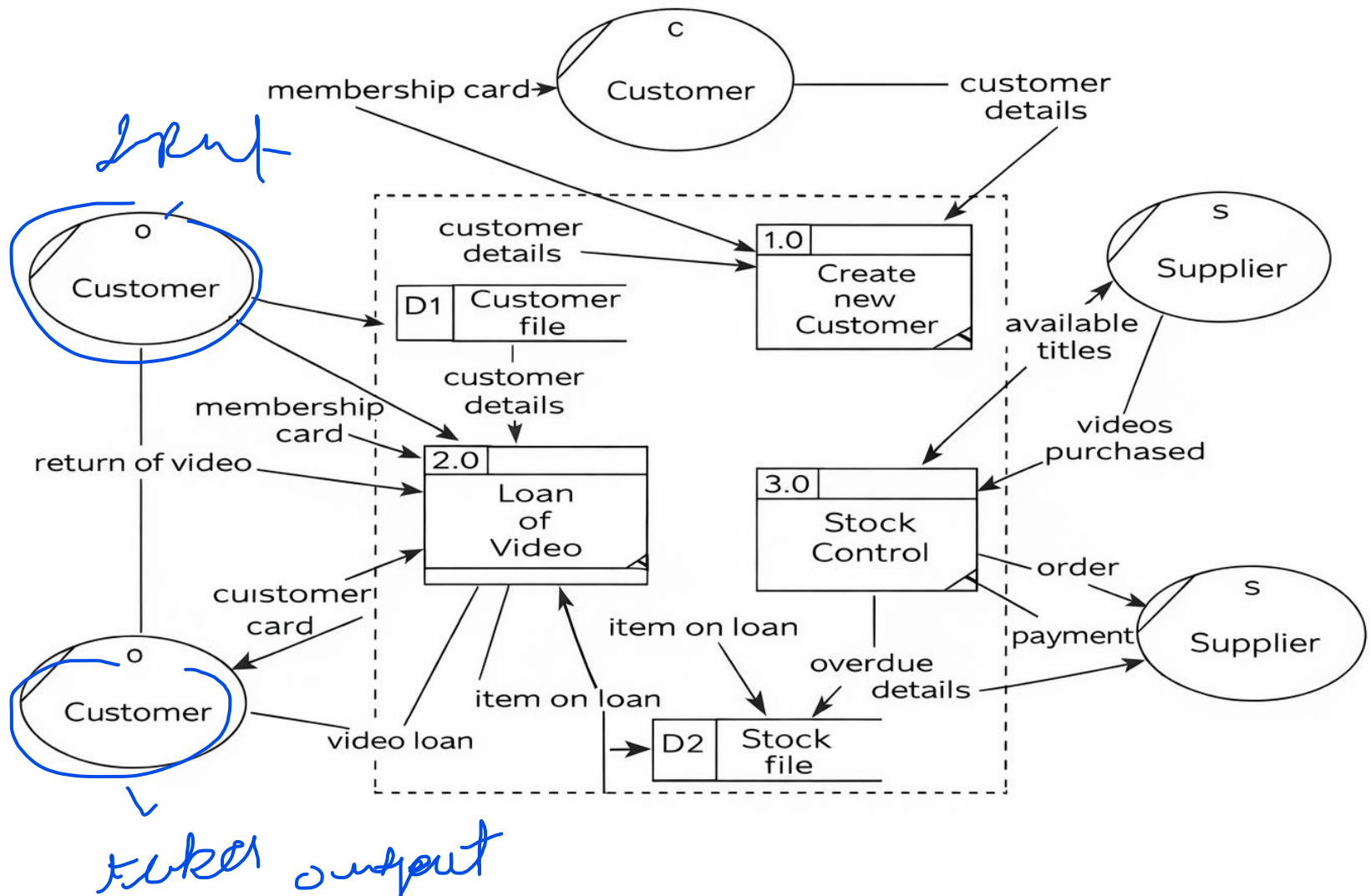
A context diagram for Video-Rental LTD



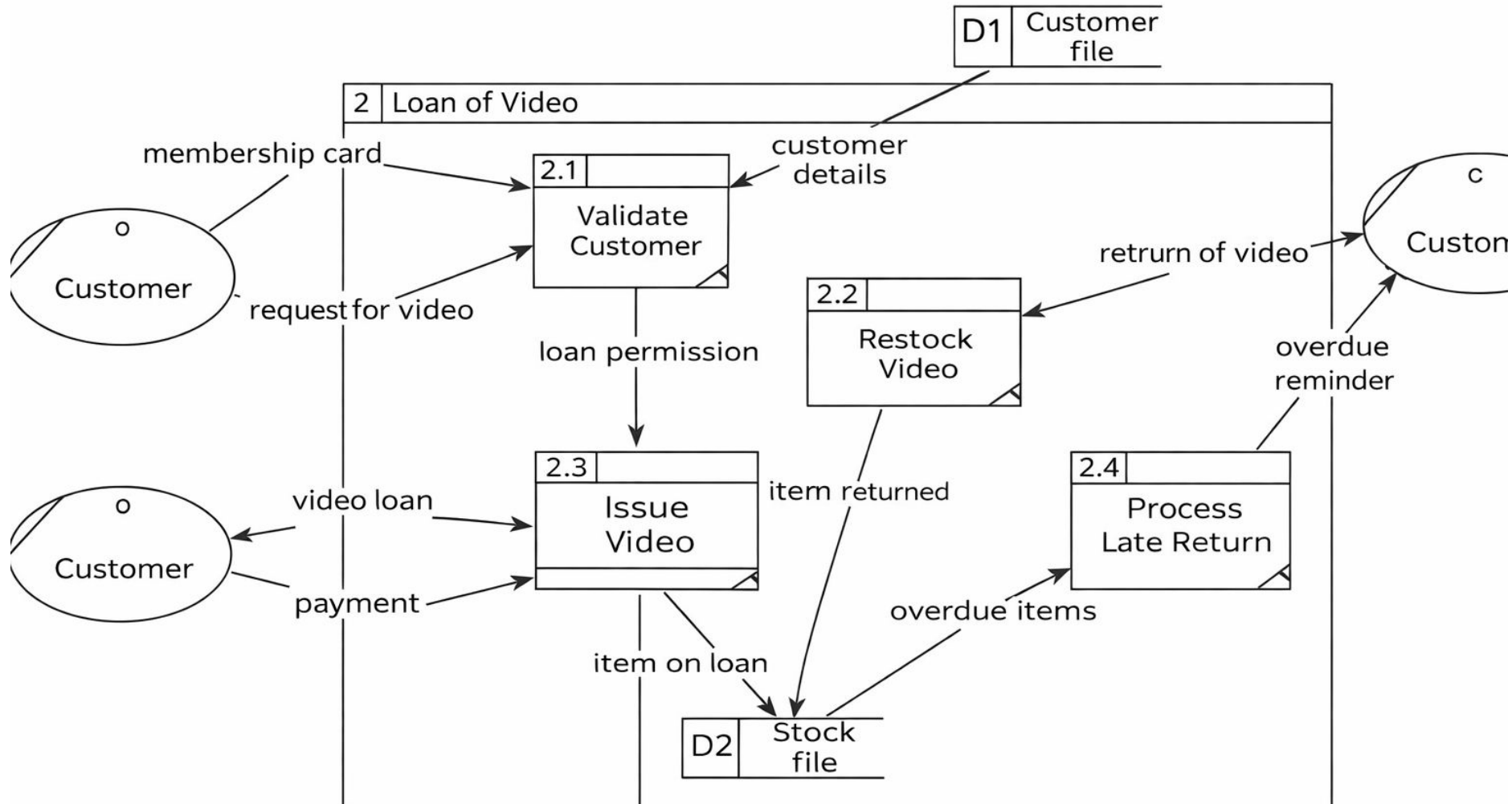
Level 1 DFD – Case Study

- Processes interact with data stores such as Customer File and Stock File.
- All external data flows balance with the context diagram.
- The system functionality becomes clear.

Level 1 DFD – Case Study



Level 2 DFD – Case Study



Common Mistakes in DFDs

- Including implementation details.
- Violating balancing rules.
- Using incorrect process names.

Benefits of Using DFDs

- Improved understanding of requirements.
- Better communication with users.
- Strong foundation for architecture design.

Summary

- Analysis modeling clarifies system requirements.
- DFDs model data movement and transformation.
- DFDs are essential tools in software analysis.